SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	N.Adi Narayana	**Reg. No.:**	192525346
Date:		**Duration:**	60 minutes

Code of Conduct

I _N.Adi Narayana(192525346)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: __Adi_____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario** ○ Carefully understand the given problem statement. ○ Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements** ○ List all functional requirements required by the proposed system.
    - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.

3. **Identify Actors and Use Cases** ○ Identify all primary and secondary actors interacting with the system.
    - ○ List the major use cases associated with each actor. ○ Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)** Develop an SRS document containing:
    - ○ Introduction
    - ○ Problem Description ○ Scope of the System ○ Functional Requirements ○ Non-Functional Requirements ○ Use Case Descriptions ○ Data Requirements ○ External Interface Requirements ○ System Features
5. **Identify Assumptions and Constraints** ○ List the assumptions considered while developing the system.
    - ○ Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency** ○ Verify whether all stakeholder requirements have been addressed. ○ Check for ambiguity, redundancy, inconsistency, and missing requirements. ○ Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and wellclassified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.

Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is wellstructured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

1. Introduction

1.1 Purpose of the Document

This Software Requirement Specification (SRS) describes the requirements of the proposed application “Customer Support Ticket Classification Using NLP and Machine Learning”. It defines what the system must do, who will use it, which data it handles and the conditions under which it operates. The document is intended to serve as a common reference for students, evaluators, designers, developers and testers during the later phases of the software development life cycle.

1.2 Project Overview

Organisations that sell products or services receive a continuous stream of customer support tickets. Each ticket is a short piece of free text in which a customer describes a problem, such as a failed payment, a forgotten password, a software error or a question about a plan. Before an agent can help, the ticket must be understood and sent to the correct support team.

The proposed system automates this first step. A customer submits a ticket through a web interface. The system processes the text using Natural Language Processing (NLP), predicts the most suitable ticket category using a Machine Learning (ML) classifier, optionally identifies the ticket priority, and routes the ticket to the relevant support team. Administrators monitor the results through a dashboard.

1.3 Role of NLP and Machine Learning

Computers cannot work directly with raw sentences, so NLP techniques are used to clean the text, split it into tokens, remove words that carry little meaning and convert the remaining words into numerical features, for example by using TF-IDF (Term Frequency–Inverse Document Frequency). A supervised ML algorithm, such as Naive Bayes, Logistic Regression or a Support Vector Machine, is then trained on historical tickets that already carry correct category labels. Once trained, the model predicts the category of new tickets in a fraction of a second and does so consistently, regardless of how many tickets arrive.

1.4 Objectives of the Document

- To state the functional and non-functional requirements of the system clearly and unambiguously.
- To identify the actors and use cases and present them in a UML use case diagram.
- To describe the data, interfaces and classification methodology required for the system.
- To record assumptions and constraints, and to propose checks that confirm the quality of the requirements.

1.5 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirement Specification
NLP	Natural Language Processing – computer techniques for working with human language
ML	Machine Learning – algorithms that learn patterns from labelled data
TF-IDF	Term Frequency–Inverse Document Frequency, a method of converting text into weighted numerical features
Ticket	A support request submitted by a customer, containing a subject and a description
Category	The class assigned to a ticket (for example, Billing and Payment)
Routing	Sending a ticket to the support team responsible for its category
REST API	A web service interface that uses HTTP requests to exchange data
UML	Unified Modeling Language

2. Problem Description

In many organisations, incoming tickets are read and categorised by support staff or by simple rule-based filters. This manual approach causes several problems, summarised in the following table.

Table 2.1: Problems with Manual Ticket Classification

Problem	Description	Effect on the Organisation
Delays in handling	Each ticket must be read before it can be forwarded. When many tickets arrive together, a queue builds up before any real support work begins.	Longer response and resolution times; lower customer satisfaction.

Incorrect categorisation	Different staff may interpret the same message differently. Tired or inexperienced staff may mislabel tickets, especially when a message mentions more than one issue.	Tickets reach the wrong team, are bounced between teams and take longer to resolve.
Repetitive work	Reading and tagging similar tickets all day is monotonous and uses skilled staff time for a low-value task.	Reduced productivity and agent fatigue.
Difficulty with large volumes	Manual effort grows with ticket count. Seasonal peaks, product launches or outages create sudden surges.	Backlogs; need to hire additional staff temporarily.
Inconsistent decisions	No fixed standard is applied by all individuals; rule-based keyword filters fail when customers use unexpected wording.	Unreliable reports and poor trend analysis.
Limited visibility	Manual tagging rarely produces structured data on ticket categories, priorities and volumes.	Managers cannot easily plan staffing or find recurring problems.

There is therefore a need for an automated, consistent and scalable method of understanding the text of a ticket and directing it to the right team. The proposed NLP and ML-based system addresses this need.

3. Scope of the System

3.1 In-Scope Features

- User registration and login for customers, support agents and administrators.
- Ticket submission through a web form (subject and description, with optional attachment reference).
- Text preprocessing: cleaning, normalisation, tokenisation, stop-word removal and stemming or lemmatisation.
- NLP feature extraction using TF-IDF.
- Automatic classification into five categories: Technical Issue, Billing and Payment, Account Access, Product or Service Inquiry and General Inquiry.
- Optional priority identification (High, Medium, Low).
- Automatic routing of the ticket to the support team that handles the predicted category.
- Ticket status tracking (New, Classified, Assigned, In Progress, Resolved, Closed).
- Storage of users, tickets, classification results, assignments and evaluation records in a database.
- Administrator dashboard with ticket counts, category distribution and model performance.

- Classification reports and model evaluation using accuracy, precision, recall, F1-score and confusion matrix.
- User, category and ticket management.

3.2 Out-of-Scope Items and Limitations

- The system classifies and routes tickets; it does not automatically answer or resolve them.
- Only English-language tickets are considered in the first version.
- Classification is limited to the five defined categories; tickets that fit none of them are expected to be assigned to General Inquiry or reviewed by an agent.
- Prediction quality depends on the quantity and quality of labelled training data; predictions may occasionally be wrong, so agents must be able to correct them.
- Voice calls, live chat transcripts and social media channels are not directly integrated.
- No special hardware, sensors or mobile-native applications are required or included.
- Integration with third-party helpdesk products is limited to optional REST API and email notification.

4. Scenario Analysis

4.1 Business Domain

The system belongs to the customer service and technical support domain. It can be used by any organisation, such as an e-commerce company, a telecom provider or a software-as-aservice business, that receives written support requests from customers and operates specialised support teams.

4.2 Stakeholders

Table 4.1: Stakeholders and Their Interests

Stakeholder	Role	Interest in the System
Customer	Submits tickets and tracks their progress	Quick acknowledgement, correct team handling the issue, visible status.
Support Agent	Resolves tickets belonging to a team	Receives only relevant tickets with predicted category and priority; can correct wrong predictions.
Administrator	Manages users, categories and monitoring	Reliable routing, dashboards, reports and control of model quality.
Support Team Manager	Plans workload of teams	Accurate ticket volume per category for staffing decisions.
Management / Business Owner	Funds and oversees support operations	Lower cost, faster resolution and improved customer satisfaction.
Developers and Testers	Build and verify the system	Clear, testable requirements.

Email / Notification Service (secondary)	External service used to send messages	Receives notification requests from the system.

4.3 Existing Challenges

The existing situation is characterised by manual reading and tagging of tickets, delays before tickets reach the right team, inconsistent labels, workload peaks, and little structured data for analysis, as described in Section 2. Customers receive slower responses and agents spend time on reassigning tickets instead of solving problems.

4.4 System Objectives

- Automatically predict the category of each new ticket from its text.
- Reduce the time between ticket submission and assignment to the correct team.
- Reduce misrouted tickets through consistent, data-driven classification.
- Identify the likely urgency of tickets so that critical issues can be handled first.
- Give administrators clear information about ticket volume, categories and model quality.
- Provide a design that can scale to larger numbers of tickets and be retrained as data grows.

4.5 Example Scenario

A customer writes: “I was charged twice for my subscription this month but I received only one invoice.” The customer submits this through the web form. The system cleans the text, extracts TF-IDF features and the classifier predicts Billing and Payment. The ticket is stored with this category, routed to the Billing team and its status becomes Assigned. The customer can view the status while a billing agent reviews the ticket. If the agent finds that the category is wrong, the agent can correct it, and the correction is stored for later model improvement. This scenario is an illustrative example only.

5. Functional Requirements

The functional requirements describe the services the system must provide. Each is identified by a unique ID (FR-xx) to support traceability to use cases.

Table 5.1: Functional Requirements

Req. ID	Requirement Name	Description
FR-01	User Registration	The system shall allow a new customer to register by providing name, email address and password. Email addresses must be unique.
FR-02	User Login and Logout	The system shall authenticate registered users and provide rolebased access for Customer, Support Agent and Administrator.
FR-03	Ticket Submission	The system shall allow a logged-in customer to submit a ticket containing a subject and a detailed description of the problem.

FR-04	Input Validation	The system shall reject empty tickets and tickets that are shorter or longer than the configured text limits and shall show a clear message.
FR-05	Ticket Text Preprocessing	The system shall clean and normalise ticket text (lower-casing, removal of punctuation, special characters, URLs and extra spaces).
FR-06	Tokenisation and Text Reduction	The system shall split the text into tokens, remove stop words and apply stemming or lemmatisation.
FR-07	NLP Feature Extraction	The system shall convert preprocessed text into numerical feature vectors using TF-IDF (or a suitable alternative such as bag-of-words counts).
FR-08	ML-Based Ticket Classification	The system shall pass the feature vector to a trained ML model (for example Naive Bayes, Logistic Regression or SVM) to classify the ticket.
FR-09	Ticket Category Prediction	The system shall assign exactly one category from: Technical Issue, Billing and Payment, Account Access, Product or Service Inquiry, General Inquiry.
FR-10	Confidence Score	The system shall record a confidence value for each prediction where the chosen model supports it, and flag low-confidence tickets for agent review.
FR-11	Priority Identification	If implemented, the system shall assign a priority (High, Medium or Low) to each ticket using trained model output and/or defined keyword rules.
Req. ID	**Requirement Name**	**Description**
FR-12	Automatic Ticket Routing	The system shall route each classified ticket to the support team mapped to its predicted category and make it visible in that team's queue.
FR-13	Ticket Status Tracking	The system shall maintain and display ticket status (New, Classified, Assigned, In Progress, Resolved, Closed) with time of each change.
FR-14	Customer Ticket View	The system shall allow a customer to view the submitted tickets, their predicted category and current status.
FR-15	Agent Ticket Handling	The system shall allow a support agent to view assigned tickets, update status and add resolution notes.
FR-16	Category Correction	The system shall allow an agent or administrator to change a wrongly predicted category and shall save the corrected label with the original prediction.
FR-17	Manual Assignment	The system shall allow an administrator to assign or reassign a ticket to a team or agent.
FR-18	Database Storage	The system shall store users, tickets, classification results, assignments and model evaluation records in a relational database.

FR-19	Administrator Dashboard	The system shall display counts of tickets by category, priority and status, and the volume over time.
FR-20	Classification Reports	The system shall generate reports on ticket classification, including category-wise counts and correction rate, which can be filtered by date.
FR-21	Model Evaluation	The system shall compute and store accuracy, precision, recall, F1-score and confusion matrix for the model on a labelled test dataset.
FR-22	User Management	The system shall allow an administrator to create, update, deactivate and assign roles to users.
FR-23	Ticket Management	The system shall allow authorised users to search and filter tickets by ID, category, status, priority and date.
FR-24	Category and Routing Configuration	The system shall allow an administrator to maintain categories and the mapping between categories and support teams.
FR-25	Notifications	The system shall send email notifications when a ticket is received and when its status changes.
FR-26	Model Retraining Support	The system shall allow an administrator to retrain the model with agent-corrected labelled tickets and record the new model version.

5.1 Requirement Prioritisation (MoSCoW)

Features are prioritised to support planning within the limited project time.

Priority	Requirements
Must have	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-09, FR-12, FR-13, FR-18, FR-19, FR-21
Should have	FR-10, FR-14, FR-15, FR-16, FR-17, FR-20, FR-22, FR-23, FR-24
Priority	**Requirements**
Could have	FR-11 (priority identification), FR-25 (notifications)
Won’t have (this version)	Multilingual classification and automated replies (see Section 16); FR-26 is planned as an extension after the basic version.

6. Non-Functional Requirements

Table 6.1: Non-Functional Requirements

ID	Category	Description
NFR-01	Performance	The system should preprocess and classify a single ticket within a few seconds under normal load. Dashboard pages should load within an acceptable time. Target values will be confirmed during testing.

NFR-02	Security	Passwords shall be stored as salted hashes; all traffic shall use HTTPS; role-based access control shall prevent customers from seeing other customers' tickets; inputs shall be sanitised against injection and scripting attacks.
NFR-03	Reliability	Ticket data shall not be lost on failure. If the classifier cannot process a ticket, it shall be saved with status New and placed in a manual review queue.
NFR-04	Usability	The interface shall be simple, consistent and understandable for non-technical customers, with clear form messages and status indicators.
NFR-05	Scalability	The design shall allow increased ticket volume, for example by separating the classification component from the web layer and using database indexing.
NFR-06	Maintainability	Code shall be modular (preprocessing, features, model, routing) and documented so that the model or text pipeline can be changed without rewriting the entire system.
NFR-07	Availability	The system should be available during business hours with planned downtime announced in advance. A high availability target (for example, 99%) will be agreed with the stakeholders.
NFR-08	Compatibility	The web interface shall work on current versions of common browsers (Chrome, Edge, Firefox, Safari) and on desktop, tablet and mobile screen sizes.
NFR-09	Classification Quality and Monitoring	Model quality shall be measured by accuracy, precision, recall and F1-score; an acceptable minimum will be agreed with stakeholders before deployment. The system shall record the rate of agent corrections and low-confidence predictions to monitor model drift.
NFR-10	Data Privacy	Personal data in tickets shall be collected only as needed, accessed only by authorised users and handled according to applicable privacy rules.
NFR-11	Auditability	Important actions (login, category correction, assignment, model retraining) shall be logged with user and time.

7. Actors and Use Cases

7.1 Actors

Table 7.1: Actors and Their Major Use Cases

Actor	Type	Description	Major Use Cases
Customer	Primary	A person who raises support requests	Register / Log In; Submit Support Ticket; Track Ticket Status; View Predicted Category and Priority
Support Agent	Primary	Staff member of a support team who resolves tickets	Log In; View Assigned Tickets; View Predicted Category and Priority; Correct Predicted Category; Update Ticket Status

Administrator	Primary	Manages and monitors the system	Log In; Manage Users; Manage Categories and Routing Rules; Assign / Reassign Ticket; View Admin Dashboard; Generate Classification Reports; Evaluate Model Performance
Email / Notification Service	Secondary	External service that delivers messages	Send Notification (triggered by the system)

The classification engine and the routing logic are internal components of the system, not actors. They are represented by the use cases Classify Ticket and Route Ticket to Support Team, which are triggered when a ticket is submitted.

7.2 UML Use Case Diagram

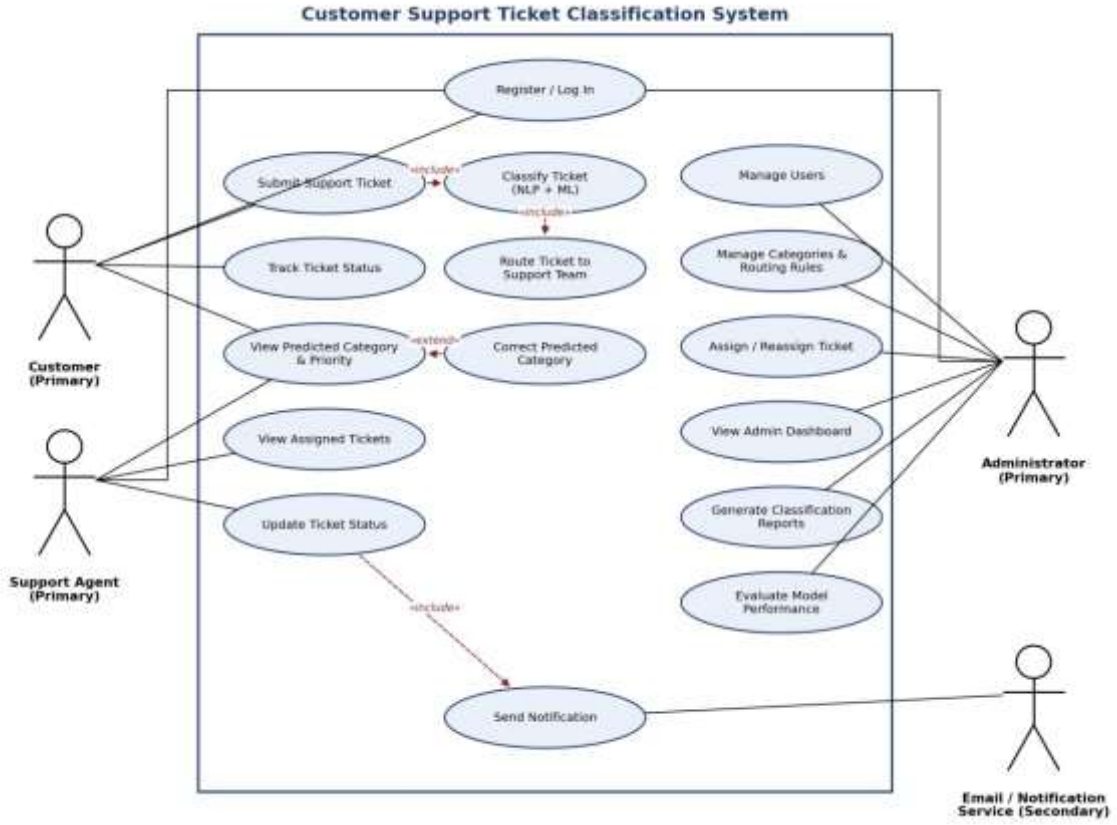


Figure 7.1: UML Use Case Diagram of the Customer Support Ticket Classification System

In the diagram, the rectangle is the system boundary. Primary actors appear on the left and right of the boundary and the secondary actor (Email / Notification Service) is at the lower right. The dashed arrow labelled «include» shows behaviour that is always executed as part of another use case: Submit Support Ticket includes Classify Ticket, Classify Ticket includes Route Ticket to Support Team, and Update Ticket Status includes Send Notification. The dashed arrow labelled «extend» shows that Correct Predicted Category is an optional extension of View Predicted Category and Priority, performed only when the agent finds the prediction incorrect.

8. Use Case Descriptions

Table 8.1: Use Case Descriptions

Use Case	Primary Actor	Description
Register / Log In	Customer, Support Agent, Administrator	A new customer creates an account; all users log in with email and password and are directed to a page based on their role.
Submit Support Ticket	Customer	The customer enters a subject and description and submits. The system validates the input, stores the ticket and starts classification. Precondition: customer is logged in.
Classify Ticket	System (triggered by Submit Support Ticket)	The system preprocesses the text, extracts TF-IDF features, predicts category (and priority if implemented) and records the classification result with a confidence value.
Route Ticket to Support Team	System (triggered by Classify Ticket)	The system looks up the team mapped to the predicted category, creates a ticket assignment and sets the status to Assigned. Low-confidence tickets go to a review queue.
View Predicted Category and Priority	Customer, Support Agent	The user opens a ticket and sees the predicted category, priority and confidence (confidence is shown to agents). Access is limited to the owner of the ticket or authorised staff.
Correct Predicted Category	Support Agent	If the prediction is wrong, the agent selects the correct category. The system stores the corrected label, updates routing and keeps the original prediction for later evaluation.
Track Ticket Status	Customer	The customer views the current status and history of a submitted ticket.
View Assigned Tickets	Support Agent	The agent sees the tickets in the queue of the agent's team, filtered by status, category or priority.
Update Ticket Status	Support Agent	The agent changes the status (for example, In Progress or Resolved) and may add notes. The system records the change and sends a notification.
Assign / Reassign Ticket	Administrator	The administrator assigns a ticket to a team or agent or moves a ticket between teams.
Manage Users	Administrator	The administrator creates, edits, deactivates users and assigns roles.
Use Case	**Primary Actor**	**Description**
Manage Categories and Routing Rules	Administrator	The administrator maintains ticket categories, priority rules and the mapping of categories to teams.
View Admin Dashboard	Administrator	The administrator views charts and counts of tickets by category, priority, status and period.

Generate Classification Reports	Administrator	The administrator creates category-wise and periodwise reports, including correction rates, and exports them.
Evaluate Model Performance	Administrator	The administrator runs the model on labelled test data and views accuracy, precision, recall, F1-score and the confusion matrix; the results are stored as an evaluation record.
Send Notification	Email / Notification Service	On request from the system, the service delivers an email to the customer or agent about ticket receipt or status changes.

8.1 Detailed Description of Key Use Case: Submit Support Ticket

Item	Details
Use Case ID / Name	UC-02 / Submit Support Ticket
Primary Actor	Customer
Preconditions	Customer is logged in; database and classifier are available.
Main Flow	1. Customer opens the ticket form. 2. Customer enters subject and description and submits. 3. System validates the input (FR-04). 4. System stores the ticket with status New. 5. System classifies the ticket (UC-Classify Ticket). 6. System routes the ticket (UC-Route Ticket). 7. System displays the ticket ID and the current status.
Alternative Flows	3a. Input invalid: system shows an error and asks the customer to correct it. 5a. Classifier unavailable: ticket stays in New status and is placed in the manual review queue.
Postconditions	Ticket is stored with a category, assignment and status; customer can track it.

9. Data Requirements

The system stores structured data in a relational database. The main entities are listed below; field names are indicative and may be refined during design.

Table 9.1: Data Entities

Entity	Purpose	Important Fields
User	Stores all people who log in to the system	user_id (PK), name, email (unique), password_hash, role (Customer / Agent / Administrator), team_id (for agents), status, created_at
Support Ticket	Stores each submitted ticket	ticket_id (PK), customer_id (FK to User), subject, description, cleaned_text (optional), status, created_at, updated_at
Entity	**Purpose**	**Important Fields**

Ticket Category	Defines the categories and the responsible team	category_id (PK), category_name, description, team_name, is_active
Classification Result	Stores the output of the model for a ticket	result_id (PK), ticket_id (FK), predicted_category_id (FK), confidence_score, predicted_priority, model_version, final_category_id (after agent correction), classified_at
Ticket Assignment	Stores to whom a ticket is routed	assignment_id (PK), ticket_id (FK), team_name, agent_id (FK, optional), assigned_by (System or administrator), assigned_at, resolution_notes
Model Evaluation Record	Stores the quality measures of each evaluated model version	evaluation_id (PK), model_name (e.g. Logistic Regression), model_version, dataset_description, accuracy, precision, recall, f1_score, confusion_matrix (stored as text/JSON), evaluated_at
Ticket Status History	Keeps the log of each status change	history_id (PK), ticket_id (FK), old_status, new_status, changed_by, changed_at
Audit Log	Stores the important system actions	log_id (PK), user_id (FK), action, timestamp

9.1 Relationships

- One User (Customer) can submit many Support Tickets.
- One Support Ticket has one current Classification Result (earlier results may be kept for history) and one Ticket Assignment (or more if reassigned).
- One Ticket Category can be linked to many Classification Results.
- One Model Evaluation Record belongs to one model version, which is referenced by Classification Results.

9.2 Training Data Requirement

The classifier requires a labelled dataset of past tickets, where each ticket text has a correct category assigned by a human. The dataset should contain a reasonable number of examples for each of the five categories and should be split into training and test portions. Tickets corrected by agents can be added later as new labelled data.

10. External Interface Requirements

10.1 User Interface

The system shall provide a browser-based interface with the following screens: registration and login; ticket submission form; customer ticket list and status page; agent queue and ticket detail page; administrator dashboard; user, category and routing management pages; report and model evaluation pages. The interface shall be responsive for desktop and mobile browsers and shall display validation and error messages clearly.

10.2 Software and Application Interfaces

Interface	Description
Web front end – back end	The browser communicates with the application server over HTTPS.
Application – classification module	The back end calls the preprocessing and classification module. This may be a library within the application or a separate service exposed through an internal REST API (for example, POST /api/classify with the ticket text, returning category, priority and confidence).
Application – database	The application connects to a relational database (such as MySQL or PostgreSQL) through a standard database driver or an ORM.
Email service	The application sends notifications through an SMTP server or an email API provider.
Optional REST API	Optional endpoints may allow other systems, such as an existing website or helpdesk tool, to submit tickets and read status. Such APIs are included only if integration is required.

10.3 Communication Interfaces

Communication shall use HTTP over TLS (HTTPS). Data exchanged through APIs shall be in JSON format.

10.4 Hardware Interfaces

No special hardware or sensors are needed. The system runs on a standard server (or cloud virtual machine), and users access it with ordinary computers or mobile devices with a web browser.

11. Classification Methodology

This section explains the proposed method in simple terms. It describes a plan; no training or testing has been carried out as part of this document, and no accuracy values are claimed.

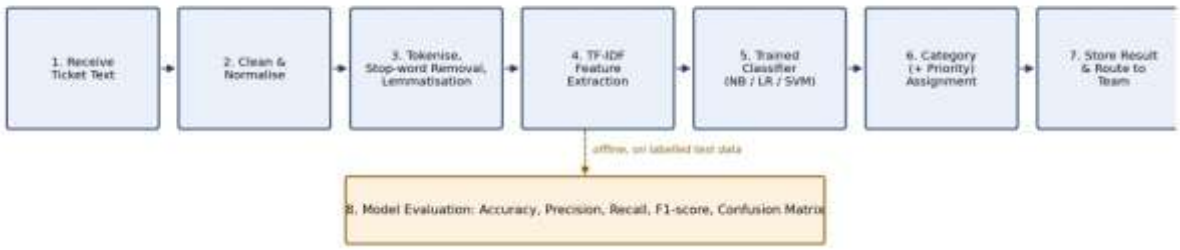


Figure 11.1: Proposed NLP and Machine Learning Classification Pipeline

11.1 Pipeline Steps

1. Receive the ticket text: the subject and description entered by the customer are joined to form one text input.

2. Clean and normalise the text: convert to lower case; remove HTML tags, URLs, special characters, extra spaces and repeated punctuation; optionally handle numbers and common abbreviations.
3. Tokenise and preprocess: split the text into words (tokens); remove stop words such as "the" or "is"; apply stemming or lemmatisation so that words like "charged" and "charging" are treated as a single form.
4. Convert text to numerical features: use TF-IDF, which gives a higher weight to words that are frequent in a ticket but rare in other tickets. Word pairs (bigrams) may be added to capture phrases such as "password reset".
5. Predict the category: give the feature vector to a trained classifier (Naive Bayes, Logistic Regression or Support Vector Machine) which outputs the most likely category and, where available, a confidence score.
6. Assign category and priority: the predicted category is attached to the ticket. If priority identification is implemented, a High, Medium or Low value is determined using a second classifier or keyword-based rules (for example, words such as "urgent", "not working" or "unable to access").
7. Store and route: the result is saved in the database and the ticket is sent to the support team mapped to the category. Tickets with low confidence are flagged for agent review.
8. Evaluate the model: using a labelled test set that was not used for training, calculate accuracy, precision, recall, F1-score and a confusion matrix.

11.2 Candidate Algorithms

Algorithm	Brief Idea	Possible Advantage	Possible Limitation
Multinomial Naive Bayes	Uses word frequencies and probability to choose the most likely category	Fast and simple; works with small datasets	Assumes words are independent
Logistic Regression	Learns weights for each feature to separate categories	Gives probability values; easy to interpret	May need tuning for many categories
Linear Support Vector Machine	Finds a boundary that best separates categories in the feature space	Often performs well on high-dimensional text	Does not directly give probabilities

The three algorithms will be compared on the same labelled data, and the one with the best evaluation results and acceptable speed will be selected. This comparison is a planned activity.

11.3 Evaluation Measures

Measure	Meaning
Accuracy	Number of correct predictions divided by the total number of predictions.
Precision	For a category, the proportion of tickets predicted as that category that really belong to it: TP / (TP + FP).

Recall	For a category, the proportion of tickets of that category that were found by the model: TP / (TP + FN).
Measure	**Meaning**
F1-score	The harmonic mean of precision and recall: 2 × (Precision × Recall) / (Precision + Recall).
Confusion matrix	A table in which rows are the actual categories and columns are the predicted categories; it shows which categories are confused with each other.

Here TP, FP and FN denote true positives, false positives and false negatives. For a multi-class problem, precision, recall and F1-score are calculated for each category and may be averaged (macro or weighted) for an overall figure. The confusion matrix will have five rows and five columns, one for each category, and will be populated only after the model is actually tested.

11.4 Handling Priority

Priority identification is optional. A simple version may use keyword rules; an extended version may train a second classifier on tickets labelled with priority. Since priority labels may be subjective, they should be reviewed by support staff before being used for decisions.

12. System Features

- Secure registration, login and role-based access for customers, agents and administrators.
- Easy web-based ticket submission with input validation.
- Automated text cleaning, tokenisation and TF-IDF feature extraction.
- ML-based prediction of ticket categories (Technical Issue, Billing and Payment, Account Access, Product or Service Inquiry, General Inquiry).
- Optional priority identification (High, Medium, Low).
- Automatic routing of tickets to the responsible support team.
- Real-time ticket status tracking for customers and agents.
- Agent ability to correct predictions, creating labelled data for improvement.
- Administrator dashboard with category, priority and status statistics.
- Classification reports and model evaluation (accuracy, precision, recall, F1-score, confusion matrix).
- User, ticket, category and routing management.
- Email notifications for ticket receipt and status updates.
- Database storage of all tickets, results, assignments and evaluation records.

13. Assumptions

Table 13.1: Assumptions

ID	Assumption	Description
A-01	Internet access	Users and the server have a reliable internet or network connection.

ID	Assumption	Description
A-02	Labelled training data	A sufficiently large and correctly labelled set of historical tickets is available or can be prepared for the five categories.
A-03	Valid ticket text	Customers write ticket descriptions in understandable English that describes the problem.
A-04	Fixed category set	The five categories are enough for the initial version and the business agrees on their definitions.
A-05	Database availability	A relational database is installed and operational whenever the system is in use.
A-06	Support teams exist	Support teams or agents responsible for each category are available to receive routed tickets.
A-07	Email service	An email service is available for sending notifications.
A-08	User devices	Users have a modern web browser.
A-09	Agent feedback	Agents are willing to correct wrong predictions so that quality can be monitored.

14. Constraints

Table 14.1: Constraints

Type	Constraint	Description
Technical	Model dependence	Prediction quality is limited by the chosen algorithm, the preprocessing and the training data. Some wrong predictions are unavoidable.
Technical	Language	The first version handles English text only.
Operational	Human review	Low-confidence tickets and corrections need agent attention; the system supports agents but does not replace them.
Economic	Limited budget	The project should use open-source tools and low-cost hosting.
Data-related	Data quantity and balance	Some categories may have fewer examples, which can reduce precision and recall for those categories. Labels may contain human error.
Privacy and Legal	Personal data	Tickets may contain personal information. Data must be protected, access must be limited, and applicable data protection rules must be followed.
Time	Project schedule	The system must be specified, designed and developed within the academic schedule, so lower priority features may be postponed (see Section 5.1).

Interface	Browser-only access	Access is through web browsers; no native mobile applications are planned.

15. Requirement Validation

Requirement validation checks that the written requirements correctly and completely describe what stakeholders need. The checks below are proposed validation activities for this SRS. They are not claimed to have been completed by stakeholders unless stated.

15.1 Validation Criteria

Criterion	Proposed Method of Checking
Completeness	Compare the requirements with the assignment instructions and the stakeholder needs in Section 4; ensure every use case has at least one functional requirement.
Consistency	Check that terms (for example, ticket, category, status) are used with the same meaning and that no two requirements conflict.
Correctness	Review each requirement with stakeholders such as support staff and the project guide to confirm that it reflects real needs.
Feasibility	Confirm that the requirements can be implemented using available tools, data and the project schedule.
Ambiguity	Check for vague words; where used (for example, “within a few seconds”), mark them for agreement on measurable values.
Traceability and testability	Map requirements to use cases and define at least one test for each.

15.2 Validation Checklist

Table 15.1: Proposed Validation Checklist

No.	Check	Status
1	All five required ticket categories are included in the requirements	Checked in this document (self-review)
2	Each functional requirement has a unique ID and clear description	Checked in this document (self-review)
3	Each use case in Section 8 is supported by at least one functional requirement	Checked in this document (self-review)
4	Each actor in the diagram has at least one use case	Checked in this document (self-review)
5	Non-functional requirements cover performance, security, reliability, usability, scalability, maintainability, availability, compatibility and classification quality	Checked in this document (self-review)
6	Quantitative non-functional targets (response time, availability, minimum model quality) are agreed	Pending – to be confirmed with stakeholders
7	Requirements reviewed by stakeholders or project guide	Pending – proposed

8	Availability and sufficiency of labelled training data confirmed	Pending – proposed
9	Redundant or conflicting requirements identified and removed	Proposed during review
10	Model performance on test data reported	Not performed – to be done during implementation
11	Privacy and legal requirements reviewed for the target organisation	Pending – proposed

16. Suggested Improvements

- **Multilingual classification:** support tickets written in regional or other languages through language detection and multilingual models.
- **Advanced NLP models:** use word embeddings or transformer-based models (for example BERT) to understand context better than TF-IDF, if sufficient data and computing resources are available.
- **Sentiment analysis:** identify angry or dissatisfied customers so that such tickets can be handled sooner.
- **Improved priority prediction:** train a priority model using urgency cues, customer type, repeated tickets and service-level agreements.
- **Chatbot integration:** suggest answers to frequent questions or collect missing information before a ticket is created.
- **Periodic model retraining:** retrain using agent-corrected tickets and monitor model drift as products and customer language change.
- **Enhanced dashboard analytics:** add trends, recurring issue detection, response-time analysis and agent workload views.
- **Multi-label classification:** allow a ticket that mentions two problems to be associated with more than one category.
- **Additional channels:** accept tickets from email, live chat and social media.

17. Conclusion

This SRS has defined the requirements of a Customer Support Ticket Classification system using NLP and Machine Learning. It describes the problem of manual ticket handling, sets out the scope and limitations, and analyses the scenario, stakeholders and objectives. Functional requirements cover submission, preprocessing, TF-IDF feature extraction, ML-based classification, optional priority identification, routing, status tracking, storage, dashboards, reporting and model evaluation. Non-functional requirements address performance, security, reliability, usability, scalability, maintainability, availability, compatibility and classification quality monitoring.

The actors, use cases and UML use case diagram show how customers, support agents, administrators and the notification service interact with the system. The data requirements, interface requirements and classification methodology provide a clear basis for database

design, architecture design and implementation. Assumptions, constraints and the proposed validation checks highlight the risks and the work that remains, such as obtaining labelled data and measuring model quality. With these requirements as a foundation, the project team can proceed to design, develop and test an efficient and scalable ticket classification application that reduces delays and misrouting and supports better customer service.